UNIVERSIDAD PONTIFICIA BOLIVARIANA
Ingeniería de Requisitos · Ingeniería de Sistemas e Informática

Trazabilidad de un requisito

Sistema integral de gestión de tripulaciones

Entrega U4A1 · Misión 9: el guardián del conocimiento · Misión 10: auditoría

Autor: Federico Márquez
Fecha: 15/10/2026
Versión del documento: 1.0 · **Rama del repositorio:** tripulaciones

Resumen

Esta entrega cierra el proceso de ingeniería de requisitos del proyecto Tripulaciones. Hice dos cosas: archivé los artefactos con sus metadatos (Misión 9) y audité la cadena completa, de la necesidad al resultado de las pruebas (Misión 10). Todo se genera con un script que lee el código, el backlog, las pruebas y git.

Criterio de la rúbrica	Qué entrego	Dónde está
Metadatos	9 artefactos con ID único, versión, estado final, autor o revisor, fecha de cierre y artefactos relacionados.	Sección 1. README de la rama y U4A1-Trazabilidad/catalogo_artefactos.json
Herramientas	Un script que valida los metadatos y genera la matriz y la auditoría, más git como control de versiones.	Sección 2. U4A1-Trazabilidad/herramienta.py
Matriz de trazabilidad	Matriz en Excel generada a partir del código del prototipo (marcas @trace), de las pruebas y del backlog.	Sección 3. U4A1-Trazabilidad/Matriz_trazabilidad.xlsx
Auditoría end-to-end (Misión 10)	Auditoría con 21 hallazgos. La cadena está completa en 8 de 18 requisitos.	Sección 4. U4A1-Trazabilidad/Auditoria_trazabilidad.md

Quién puede entender el proyecto leyendo el repositorio. El README de la rama cuenta el contexto, las necesidades, los requisitos, la validación, la cadena de trazabilidad y las decisiones del CCB, y enlaza cada artefacto. Esa fue la prueba que usé para decidir qué faltaba.

Misión 9: el guardián del conocimiento

1.1 Metadatos de cada artefacto

Cada artefacto tiene los seis metadatos mínimos. Están en un solo archivo, U4A1-Trazabilidad/catalogo_artefactos.json, y el README de la rama muestra la misma tabla (se genera, no se escribe a mano).

ID	Artefacto	Versión	Estado final	Autor o revisor	Fecha de cierre	Artefactos relacionados
TRI-PBL-001	Product Backlog	1.0	Aprobado, 1 iteración	Federico Márquez (autor). Revisor: Pendiente (docente)	13/10/2026	TRI-RFC-002, TRI-PRO-003, TRI-MTZ-007
TRI-RFC-002	Solicitudes de cambio	1.0	Entregado, 1 iteración. Decisiones tomadas en TRI-CCB-006	Federico Márquez (autor). Revisor: Pendiente (docente)	13/10/2026	TRI-PBL-001, TRI-CCB-006, TRI-DOC-005
TRI-PRO-003	Prototipo de baja fidelidad	0.2	Validado con 30 pruebas automáticas; 7 defectos abiertos (DEF-01, DEF-06 a DEF-11)	Federico Márquez (autor). Revisor: Pendiente (docente)	13/10/2026	TRI-PBL-001, TRI-PRU-004, TRI-DOC-005, TRI-MTZ-007
TRI-PRU-004	Pruebas del prototipo	1.0	30 de 30 aprobadas	Federico Márquez (autor). Revisor: Pendiente (docente)	14/10/2026	TRI-PRO-003, TRI-PBL-001, TRI-MTZ-007
TRI-DOC-005	Prototipado y CCB (U3A2)	1.0	Entregado, 1 iteración	Federico Márquez (autor). Revisor: Pendiente (docente)	14/10/2026	TRI-PRO-003, TRI-PRU-004, TRI-CCB-006, TRI-RFC-002
TRI-CCB-006	Decisiones del CCB	1.0	Reunión simulada del 13/10/2026; 6 RFC aprobadas, 4 sin aprobar	Federico Márquez (autor). Revisor: Pendiente (docente)	13/10/2026	TRI-RFC-002, TRI-DOC-005, TRI-MTZ-007
TRI-MTZ-007	Matriz de trazabilidad	1.0	Generada por script; auditada en TRI-AUD-008	Federico Márquez (autor). Revisor: Pendiente (docente)	15/10/2026	TRI-PBL-001, TRI-PRO-003, TRI-PRU-004, TRI-CCB-006, TRI-AUD-008, TRI-SCR-009
TRI-AUD-008	Auditoría de trazabilidad	1.0	Realizada; hallazgos abiertos con responsable	Federico Márquez (autor). Revisor: Pendiente (docente)	15/10/2026	TRI-MTZ-007, TRI-SCR-009, TRI-PBL-001
TRI-SCR-009	Herramienta de trazabilidad	1.0	Operativa	Federico Márquez (autor). Revisor: Pendiente (docente)	15/10/2026	TRI-MTZ-007, TRI-AUD-008, TRI-CCB-006

Cómo se asignan los metadatos

- **ID único.** Formato PROYECTO-TIPO-NNN (por ejemplo TRI-PRO-003). TIPO es PBL (backlog), RFC, PRO (prototipo), PRU (pruebas), DOC (informe), CCB (acta), MTZ (matriz), AUD (auditoría) o SCR (script). El número no se reutiliza.
- **Versión.** 1.0 para lo entregado una vez. El prototipo está en 0.2 porque tuvo dos iteraciones (la 0.1 se conserva como evidencia en pruebas/).
- **Estado final.** Dice cómo terminó el artefacto, no solo que existe. Ejemplo: «Validado con 30 pruebas automáticas; 7 defectos abiertos».
- **Autor o revisor.** El autor soy yo. El revisor queda pendiente: lo asignará el docente.
- **Fecha de cierre.** La del último cambio. Coincide con la fecha de confirmación en git para los artefactos ya subidos.

- **Artefactos relacionados.** Los IDs de los artefactos de los que depende o que dependen de él. El script comprueba que cada ID exista.

1.2 Organización del repositorio

Una carpeta por tarea del curso. Dentro de cada carpeta está todo lo que esa tarea produjo.

Carpeta	Tarea	Qué guarda	Artefactos
Backlog/	Product Backlog	Backlog de 18 ítems (HU-01 a HU-12, RNF-01 a RNF-06) en el archivo de ReqView (.reqw, fuente) y en HTML	TRI-PBL-001
U3A1-RFC/	U3A1 · RFC	10 solicitudes de cambio con su análisis de impacto (PDF y Word)	TRI-RFC-002
U3A2-Prototipado/	U3A2 · Prototipado y CCB	Informe; prototipo/ (con marcas @trace); pruebas/ (pruebas, resultados, capturas y evidencia de la versión 0.1); trazabilidad/ (decisiones del CCB y matriz de cambios aprobados)	TRI-PRO-003, TRI-PRU-004, TRI-DOC-005, TRI-CCB-006
U4A1-Trazabilidad/	U4A1 · Trazabilidad	Informe, herramienta, necesidades, catálogo de metadatos, matriz en Excel y auditoría	TRI-MTZ-007, TRI-AUD-008, TRI-SCR-009
README.md	Índice	Contexto, necesidades, requisitos, validación, trazabilidad y decisiones del CCB; tabla de metadatos	Todos

El README de la rama sigue la estructura de las demás ramas del repositorio de referencia (tabla de versiones y secciones de contexto a gestión de cambios). Le agregué la columna de artefactos relacionados, que la tabla del otro repositorio no tiene, y las carpetas por tarea.

Herramientas

2.1 Gestión automatizada de artefactos

La herramienta es U4A1-Trazabilidad/herramienta.py (Python, usa openpyxl). Se ejecuta con un comando y no pide editar nada a mano:

python3 U4A1-Trazabilidad/herramienta.py todo

Subcomando	Qué hace
catalogo	Comprueba que cada artefacto tenga los seis metadatos, que sus archivos existan, que los artefactos relacionados existan y que estén en git. Actualiza la tabla del README.
matriz	Genera Matriz_trazabilidad.xlsx (y el CSV y el Markdown de los cambios aprobados) leyendo el código, el backlog, las pruebas y las decisiones del CCB.
auditoria	Recorre la cadena necesidad → requisito → prueba → código → resultado y escribe Auditoria_trazabilidad.md con las rupturas.
todo	Hace los tres pasos en ese orden.

Los datos de entrada son archivos del repositorio, no copias: el código del prototipo, el archivo .reqw del backlog (ReqView), los resultados de las pruebas y tres JSON pequeños (necesidades, decisiones del CCB y catálogo). Si cambia una fuente, se vuelve a ejecutar el comando y todo se actualiza.

2.2 Control de versiones

- Todo el proyecto está en git, en el repositorio FEDERICOMAR79/Trazabilidad_requisitos. El proyecto Tripulaciones está en la rama tripulaciones.
- Cada entrega se confirma en un commit propio con un mensaje que dice qué cambió. El historial de un archivo se consulta con git log --oneline -- ruta.
- La herramienta revisa que cada archivo del catálogo esté bajo git y marca un hallazgo si no lo está. Por eso se ejecuta después de git add y antes del commit.
- El versionado de cada artefacto (campo versión) es independiente del historial de git: git guarda cada cambio; la versión marca los hitos entregados.

Matriz de trazabilidad

3.1 Cómo se genera a partir del código

La matriz no se escribe a mano. Cada función del prototipo lleva un comentario que dice qué requisito implementa:

```
/* @trace HU-08 RNF-02 */ function abrirConfirmar(...) { ... }
```

El script lee esas marcas, las cruza con las pruebas (cada prueba declara el requisito que verifica) y con el backlog, y construye la matriz. Si alguien cambia el código y borra la marca, la matriz y la auditoría lo reflejan al regenerarlas.

3.2 Qué contiene Matriz_trazabilidad.xlsx

Hoja	Contenido
Matriz principal	Una fila por requisito (18): necesidad, actor, prioridad, código, casos de prueba, resultado, estado, RFC aprobadas y notas.
Trazabilidad visual	Requisitos contra necesidades y funciones del código, con ✓ y el porcentaje de cobertura de cada columna.
Resumen y métricas	Totales y coberturas con fórmulas de Excel sobre la matriz principal. Más la lista de requisitos sin código.
Registro de RFCs	Las 10 solicitudes con la decisión del CCB, votos, plazo y responsable.
Catálogo de artefactos	Los metadatos de los 9 artefactos.
Auditoría	Los hallazgos de la Misión 10.

3.3 Vista de la matriz

Req.	Título	Prior.	Necesidades	Código	Pruebas	Resultado	Estado
HU-01	Registrar y actualizar tripulantes	Alta	NEG-02, NEG-04	—	—	Sin prueba	Aprobado
HU-02	Registrar y actualizar tripulaciones	Alta	NEG-02, NEG-04	—	—	Sin prueba	Aprobado
HU-03	Registrar y actualizar vuelos	Alta	NEG-02, NEG-04	—	—	Sin prueba	Aprobado
HU-04	Consultar vuelos asignados	Alta	NEG-04	vMisVuelos	P15, P16	Pasado 2/2	Verificado
HU-05	Consultar características de un vuelo	Alta	NEG-04	fichaVuelo, vVueloOperador, vTodos, vVueloTripulante	P18	Pasado 1/1	Verificado
HU-06	Registrar resultado de simulación	Alta	NEG-01, NEG-02, NEG-05	vResultado, ligarResultado	P21, P22	Pasado 2/2	Verificado
HU-07	Recomendar tripulación según escenario más aproximado	Alta	NEG-03	distancia, masCercano, ranking, vVueloOperador	P02, P03, P10, P11, P12, P23	Pasado 6/6	Verificado
HU-08	Asignar tripulación a un vuelo	Alta	NEG-03	vVuelos, vVueloOperador, abrirConfirmar	P01, P04, P05, P06, P07, P08, P13, P14	Pasado 8/8	Verificado
HU-09	Alertar simulación anual pendiente	Media	NEG-01	—	—	Sin prueba	Aprobado
HU-10	Gestionar catálogo de escenarios	Media	NEG-02, NEG-05	—	—	Sin prueba	Aprobado
HU-11	Reportes históricos de desempeño	Baja	NEG-01	—	—	Sin prueba	Aprobado
HU-12	Notificación de cambio de asignación	Baja	NEG-04	—	—	Sin prueba	Aprobado

Req.	Título	Prior.	Necesidades	Código	Pruebas	Resultado	Estado
RNF-01	Seguridad basada en roles	Alta	NEG-04	NAV, permitido	P17	Pasado 1/1	Verificado
RNF-02	Auditoría de asignaciones	Alta	NEG-03	abrirConfirmar, vAuditoria	P09	Pasado 1/1	Verificado
RNF-03	Disponibilidad continua	Media	NEG-04	—	—	Sin prueba	Aprobado
RNF-04	Rendimiento en consultas	Media	NEG-04	—	—	Sin prueba	Aprobado
RNF-05	Integridad de la sincronización con el simulador	Media	NEG-05	—	—	Sin prueba	Aprobado
RNF-06	Usabilidad móvil	Baja	NEG-02	Estilos para pantallas móviles	P24, P25, P29	Pasado 3/3	Verificado

Los resultados salen de las 30 pruebas automáticas (30 de 30 aprobadas). Ocho requisitos están verificados y diez están aprobados pero sin construir en el prototipo.

Misión 10: auditoría de trazabilidad

4.1 Alcance y método

Audité la cadena **necesidad de negocio → requisito → caso de prueba → código → resultado** para los 18 requisitos del backlog. Para cada eslabón, el script comprueba que exista y que apunte al siguiente. También revisa las RFC aprobadas contra el backlog y las pruebas, los metadatos y el control de versiones. Una ruptura es cualquier eslabón que falta, que apunta a algo que no existe o que se contradice con otra fuente.

4.2 Resultado

Medida	Valor
Requisitos auditados	18
Con la cadena completa	8 de 18: HU-04, HU-05, HU-06, HU-07, HU-08, RNF-01, RNF-02, RNF-06
Con necesidad asociada	18 de 18
Con código en el prototipo	8 de 18
Con pruebas, todas aprobadas	8 de 18
Hallazgos	21 en total: 10 abiertos y 11 cerrados o aceptados
Severidad de los abiertos	1 alto, 8 medio, 1 bajo

Lectura. Todos los requisitos tienen una necesidad: no hay requisitos huérfanos. Los ocho que tienen código también tienen pruebas aprobadas. Los diez sin código son los que quedaron fuera del alcance del prototipo; sus rupturas están aceptadas con motivo. Lo que sí es una ruptura real son los casos de prueba de las RFC aprobadas, que existen solo en el papel, y el backlog, que no recoge todavía las decisiones del CCB.

4.3 Hallazgos abiertos

ID	Ruptura	Elemento	Detalle	Sev.	Responsable	Acción
AUD-01	RFC → prueba	RFC-A3	3 casos de prueba propuestos sin implementar (TC-HU08-08, TC-HU08-09, TC-RNF01-05).	Alto	QA Lead	Escribirlos al construir el cambio. Plazo: Extraordinaria: antes del Sprint 4 (hasta 16/10/2026).
AUD-02	Artefacto ausente	M-02	El Vision Board (fuente de las necesidades NEG-01 a NEG-05) no está en el repositorio propio.	Medio	Autor	Agregar el Vision Board como artefacto TRI-VIS y enlazarlo desde las necesidades.
AUD-03	Backlog desactualizado	M-03	El backlog (v1.0) no refleja las decisiones del CCB del 13/10/2026 (por ejemplo, el promedio de 3 simulaciones de RFC-B1 y el motivo obligatorio de RFC-A3).	Medio	Analista de requisitos	Publicar backlog v1.1 con los criterios de aceptación de las RFC aprobadas.
AUD-04	Código → requisito	vEvaluacion	La función vEvaluacion del prototipo no tiene marca @trace: implementa una pantalla sin requisito en el backlog (consulta de escenarios y evaluación del piloto, que el caso describe).	Medio	Analista de requisitos	Agregar una historia de usuario al backlog para la consulta de escenarios y marcar la función.
AUD-05	RFC → prueba	RFC-A1	4 casos de prueba propuestos sin implementar (TC-HU03-05, TC-HU07-10, TC-HU07-11, TC-HU10-04).	Medio	QA Lead	Escribirlos al construir el cambio. Plazo: Sprint 4 (26/10/2026 – 06/11/2026).
AUD-06	RFC → prueba	RFC-A2	3 casos de prueba propuestos sin implementar (TC-HU08-07, TC-HU12-03, TC-RNF02-04).	Medio	QA Lead	Escribirlos al construir el cambio. Plazo: Sprint 5 (09/11/2026 – 20/11/2026).

ID	Ruptura	Elemento	Detalle	Sev.	Responsable	Acción
AUD-07	RFC → prueba	RFC-C4	2 casos de prueba propuestos sin implementar (TC-HU09-04, TC-HU06-05).	Medio	QA Lead	Escribirlos al construir el cambio. Plazo: Sprint 4 (26/10/2026 – 06/11/2026).
AUD-08	RFC → prueba	RFC-B1	6 casos de prueba propuestos sin implementar (TC-HU06-03, TC-HU07-12, TC-HU07-14, TC-HU08-10, TC-HU11-05, TC-RNF05-03).	Medio	QA Lead	Escribirlos al construir el cambio. Plazo: Sprints 4 a 5 (26/10/2026 – 20/11/2026).
AUD-09	RFC → prueba	RFC-C2	4 casos de prueba propuestos sin implementar (TC-HU07-13, TC-HU08-11, TC-REG-01, TC-HU11-06).	Medio	QA Lead	Escribirlos al construir el cambio. Plazo: Sprints 4 a 6 (26/10/2026 – 04/12/2026).
AUD-14	Prueba → requisito	P19, P20, P26, P27, P28, P30	Pruebas sin requisito del backlog: consulta de escenarios (P19, P20) y pruebas transversales de defectos y calidad (P26 a P28, P30).	Bajo	QA Lead	Relacionar P19 y P20 con la historia nueva; las demás quedan como pruebas de regresión.

4.4 Hallazgos cerrados o aceptados

ID	Ruptura	Elemento	Estado	Motivo
AUD-10	Contradicción entre fuentes	M-01	Cerrado	El caso de estudio dice que el resultado nuevo elimina el anterior; el criterio de aceptación de HU-06 lo conserva en el historial. RFC-B1 adopta la versión con historial.
AUD-11	Requisito → código	HU-01 Registrar y actualizar tripulantes	Aceptado (fuera de alcance)	Registro administrativo; no cambia la decisión de asignación que valida el prototipo (U3A2, sección 7.1).
AUD-12	Requisito → código	HU-02 Registrar y actualizar tripulaciones	Aceptado (fuera de alcance)	Registro administrativo; fuera del alcance del prototipo (U3A2, sección 7.1).
AUD-13	Requisito → código	HU-03 Registrar y actualizar vuelos	Aceptado (fuera de alcance)	Registro administrativo; fuera del alcance del prototipo (U3A2, sección 7.1).
AUD-15	Requisito → código	HU-09 Alertar simulación anual pendiente	Aceptado (fuera de alcance)	Alerta de simulación anual; requiere calendario y notificaciones (RFC-A3, CCB 13/10/2026).
AUD-16	Requisito → código	HU-10 Gestionar catálogo de escenarios	Aceptado (fuera de alcance)	Gestión del catálogo; el prototipo trae escenarios fijos.
AUD-17	Requisito → código	HU-11 Reportes históricos de desempeño	Aceptado (fuera de alcance)	Reportes históricos; dependen del histórico de RFC-B1.
AUD-18	Requisito → código	HU-12 Notificación de cambio de asignación	Aceptado (fuera de alcance)	Notificaciones; dependen de RFC-A2 (Sprint 5).
AUD-19	Requisito → código	RNF-03 Disponibilidad continua	Aceptado (fuera de alcance)	Disponibilidad 24/7; no medible en un prototipo local.
AUD-20	Requisito → código	RNF-04 Rendimiento en consultas	Aceptado (fuera de alcance)	Rendimiento; sin backend no hay tiempos reales que medir.
AUD-21	Requisito → código	RNF-05 Integridad de la sincronización con el simulador	Aceptado (fuera de alcance)	Sincronización con el simulador; el simulador es externo y el prototipo no se conecta.

4.5 Conclusión y plan de cierre

La cadena está sana en el núcleo que se construyó (asignación, consulta y resultados) y rota donde todavía no hay construcción. Para cerrar el ciclo de los requisitos propongo, en este orden:

- Antes del 16/10/2026: escribir y ejecutar los tres casos de prueba de RFC-A3 (hallazgo de severidad alta) y confirmar que bloquean la asignación.
- Publicar el backlog v1.1 con los criterios de las RFC aprobadas (AUD-03) y agregar la historia de consulta de escenarios (AUD-04).

- Agregar el Vision Board al repositorio como artefacto TRI-VIS (AUD-02).
- En cada sprint, convertir los casos de prueba propuestos en pruebas automáticas y volver a ejecutar la herramienta: los hallazgos de RFC → prueba deben bajar a cero.

Notas y supuestos

- El Vision Board propio no está en el repositorio. Las cinco necesidades (NEG-01 a NEG-05) las tomé del Vision Board publicado en la rama tripulaciones del repositorio de referencia y las resumí. La relación de cada necesidad con los requisitos es análisis mío. Queda registrado como hallazgo AUD-02.
- Los casos de prueba que citan las RFC (TC-HU07-10 y similares) son propuestas del documento de la unidad 3. Todavía no existen como pruebas ejecutables; la auditoría los cuenta como ruptura.
- Las decisiones del CCB vienen de una reunión simulada en la que asumí los cinco roles. Las fechas y los plazos son propuestas para el ejercicio.
- La prueba con usuarios del informe U3A2 es una simulación con personas ficticias. Las pruebas automáticas, en cambio, sí se ejecutaron.
- El prototipo cubre 8 de los 18 requisitos. Los demás están marcados como fuera del alcance con su motivo.
- El revisor de cada artefacto queda pendiente hasta que lo asigne el docente.